



University of Camerino

SCHOOL OF SCIENCE AND TECHNOLOGY

Master's degree in Computer Science (LM-18)

Support for the design and deployment of IoT scenarios through Agentic AI

Author

Damiano Buzzo

Advisor

Prof. Massimo Callisto De Donato

Matricula 131989

Coadvisor

Prof. Stefàn Olafsson

A.Y. 2025/2026

Contents

1	Introduction	11
1.1	Problem statement	11
1.2	Proposed solution	11
1.3	Objectives	12
1.4	Thesis structure	12
2	Background	13
2.1	IoT	13
2.1.1	Sensing layer and physical metrics	13
2.1.2	Networking layer	14
2.1.3	Data analytics and processing	14
2.2	Digital Twin	15
2.2.1	Digital twin model and lifecycle categories	15
2.2.2	Symbiotic Connection to IoT	15
2.2.3	Intelligent Digital Twins	16
2.3	SAIBA framework	17
2.3.1	Stages of the SAIBA architecture	17
2.3.2	Integration of reactive behaviour	17
2.3.3	Application of the framework	17
2.4	Agentic AI	18
2.4.1	Conceptual taxonomy and differences	18
2.4.2	Core components	20
2.4.3	Types of architectures	20
2.5	Agents interoperability protocols	22
2.5.1	Model Context Protocol (MCP)	23
2.5.2	Agent To Agent(A2A)	24
2.5.3	Agent Communication Protocol (ACP)	25
2.5.4	Agent Network Protocol (ANP)	26
2.5.5	Structural patterns and combinations	26
2.6	Agentic AI frameworks	27
2.6.1	LangGraph	27
2.6.2	LangChain	28
2.6.3	LLamaIndex	28
2.6.4	Microsoft Agent Framework	29

2.6.5	CrewAI	29
2.6.6	OpenAI Agents SDK	29
2.6.7	Comparative evaluation	30
3	Literature Review	31
3.1	Data collection methodologies	32
3.1.1	Inclusion and exclusion criteria	32
3.2	Discussion	32
3.2.1	Natural Language Intent and Architectural Hierarchy	32
3.2.2	Deployment Automation and Workflow Resilience	33
3.2.3	Context Tracking and Autonomous Reasoning	34
3.3	Key findings and gaps	34
3.3.1	Key findings	34
3.3.2	Identified gaps	34
4	Architectural overview	37
4.1	High-level architecture	37
4.2	Interaction and Intent Planning Layer	37
4.3	Behavior Planning and Specialization Layer	38
4.4	Behavior Realization and Execution Layer	38
4.5	Platform and Services Layer	39
4.6	Technologies used	39

List of Figures

2.1	Pavatar architecture[6]	13
2.2	Digital twin lifecycle[9]	15
2.3	Front Running Simulation (FRS)[9]	16
2.4	SAIBA architecture[5]	17
2.5	Comparative illustration of AI Agent versus Agentic AI	19
2.6	ReAct Single-Agent architecture[14]	21
2.7	Supervisor/hierarchical architecture[14]	22
2.8	MCP architecture[16]	24
2.9	A2A architecture[16]	25
2.10	MCP+A2A integration[15]	27
4.1	System High-level architecture	38

List of Tables

2.1	Taxonomy of AI Paradigms	20
2.2	Comparative Evaluation of Frameworks	30
3.1	Comparative evaluation of literature approaches based on system characteristics.	33

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

1. Introduction

The rapid evolution of the Internet of Things (IoT) has fundamentally transformed modern infrastructure, seamlessly interconnecting physical assets with cyber systems through advanced sensing, communication, and data [1]. By serving as a bridge between the physical world and cyberspace, IoT enables continuous data gathering, real-time feedback, and fine-grained measurement of physical assets in large-scale networks[2].

As these ecosystems expand, they increasingly rely on Digital Twin (DT) technology to organize and manage system information. A Digital Twin serves as a comprehensive software representation of an individual physical object, capturing its properties, conditions, and behaviors across its entire [3]. This symbiotic connection relies heavily on the IoT as its backbone; the constant flow of real-time sensory data is required to synchronize the physical entity with its virtual counterpart, enabling advanced simulation, predictive maintenance, and operational optimization. The integration of these two domains forms the essential foundation for next-generation intelligent [4].

1.1 Problem statement

Despite the theoretical capabilities of Intelligent Digital Twins, orchestrating the deployment and configuration of complex IoT infrastructures remains a significant bottleneck. The rapid expansion of IoT has introduced substantial operational complexity. The heterogeneity of devices, diverse underlying technologies, and a lack of universal standardization pose critical challenges for system integration[1].

Currently, there is a substantial semantic gap between an operator's abstract, high-level intent and the low-level, field-specific actuations required across these heterogeneous platforms. Conventional programmatic approaches lack the proactivity and autonomous reasoning required to bridge this gap and adapt to dynamic environmental changes without constant human supervision.

1.2 Proposed solution

To address these challenges, this thesis explores the integration of Agentic Artificial Intelligence within the lifecycle of IoT systems. The primary focus of this research is to investigate how autonomous AI agents can streamline the transition from high-level design requirements to functional deployments by leveraging Digital Twin models.

By treating Digital Twins as standardized abstractions of physical assets, this study aims to develop a methodology where AI agents can independently interpret system specifications, query available digital models, and orchestrate the necessary configurations on target IoT platforms. At the core of this framework, Large Language Models (LLMs) serve as the reasoning engine. By utilizing Chain-of-Thought (CoT) prompt-

ing, the system can autonomously generate a series of intermediate reasoning steps to decompose complex, multi-step deployment problems into executable sub-tasks.

To effectively map abstract goals to physical actions, this methodology draws inspiration from the Situation, Agent, Intention, Behavior, Animation (SAIBA) framework[5], which structurally separates intent planning from behavior realization. Furthermore, by incorporating multimodal reasoning capabilities, these agentic systems can process not just text, but complex visual and structural data from the Digital Twin to ground their actions in the reality of the physical environment.

1.3 Objectives

The main aim of this project is to facilitate the deployment of a complex IoT infrastructures required to enable advanced digital twin scenarios. The specific contributions of this thesis include:

- **RQ1** (Architectural Design): What hierarchical orchestration architecture can best empower IoT architects to translate abstract, natural language requirements into actionable, agnostic IoT deployment strategies?
- **RQ2** (Implementation Strategy): How can LLM-based reasoning engines be structured within a multi-agent ecosystem to reliably coordinate complex, multi-step deployment workflows in dynamic IoT environments?
- **RQ3** (Empirical Validation): To what extent does the proposed agentic deployment loop support the operational reliability and deployment consistency of an IoT infrastructure when tested against a concrete demonstration use case?

1.4 Thesis structure

The remainder of this thesis is structured as follows:

- Chapter 2 (**Background**): Provides a comprehensive review of the foundational technologies, including the layers of IoT architecture, Digital Twin lifecycles, the SAIBA framework, and the conceptual taxonomy of Agentic AI paradigms
- Chapter 3 (**Literature review**): Discusses already existent approaches to the problem, comparing them using characteristic we are looking for in the project
- Chapter 4 (**Architecture and Methodology**): Details the proposed agentic framework, mapping the transition from the Intent Planning stage to Behavior Realization
- Chapter 5 (**Implementation**): Discusses the technical realization of the proposed system using modern multi-agent frameworks.
- Chapter 6 (**Evaluation**): Presents the experimental setup and analyzes the performance of the autonomous deployment loop.
- Chapter 7 (**Conclusion**): Summarizes the findings and outlines directions for future research.

2. Background

2.1 IoT

The Internet of Things (IoT) has evolved from providing purely surveillance-centric functions to acting as a comprehensive information framework for industrial processes[6]. It serves as a bridge between the physical world and cyberspace, enabling fine-grained sensing of objects, continuous data gathering, and real-time feedback or control. Today, the significance of IoT is emphasized by its role in building "Industry 4.0" and the "Industrial Internet" where billions of connected things provide smart services. In the following subsection will be examined all the layers of a typical IoT architecture, as described in 2.1

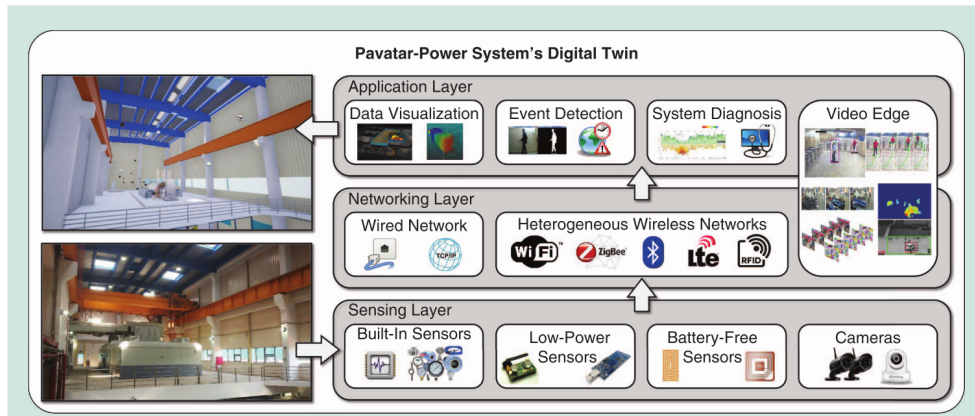


Figure 2.1: Pavatar architecture[6]

2.1.1 Sensing layer and physical metrics

The foundation of modern IoT scenarios is the ability to monitor diverse physical metrics through non-intrusive sensing[6]. In industrial environments, key metrics include:

- **Operational Health:** Metrics such as temperature, pressure, vibration, noise, and rotation speed are critical for monitoring rotating machines
- **Advanced Metrics:** Specialized sensing includes eccentricity (displacement of a rotating center), which is vital for synchronizing compensators in power station management
- **Material and Target Identification:** Techniques like TagScan utilize the differences in Radio Frequency (RF) signals when traversing different liquids or solids to classify materials and image targets.

- **Battery-Free Sensing:** The development of wireless and battery-free sensing, particularly through RFID, allows for low-cost and robust monitoring in complex environments

Furthermore, the sensing layer has expanded to include visual sensing from intensive networked videos. Due to the high bandwidth and low-latency requirements of video data, modern IoT systems increasingly utilize collaborative edge computing to process these intensive streams near the source rather than relying on the cloud. This necessitates the implementation of embedded deep learning models, such as Convolutional Neural Networks (CNNs), directly onto resource-constrained edge devices to provide real-time visual perception and target identification[7].

2.1.2 Networking layer

The networking layer is responsible for the efficient transmission of gathered data between heterogeneous devices, often operating within complex and congested frequency environments[6].

A primary challenge in modern IoT connectivity is the coexistence of incompatible protocols, such as Wi-Fi, ZigBee, and Bluetooth, which frequently occupy the same spectral bands. To address this, current research focuses on Cross-Technology Communication (**CTC**), which enables direct interaction between these disparate devices by utilizing side-channel information like transmission timing, beacon shifting, and RSSI amplitude without requiring hardware modifications.

Furthermore, the standardization of technologies such as Narrow-Band IoT (NB-IoT) has become a central strategy for establishing low-power, wide-area networks (LPWAN) capable of supporting massive device density. These advancements are complemented by resource-saving innovations, including parallel backscatter transmissions and software-defined wireless protocols, which significantly reduce energy consumption and facilitate the deployment of battery-free sensing nodes in large-scale scenarios.[8]

2.1.3 Data analytics and processing

The analytics layer transforms **raw signals into actionable knowledge** through sophisticated signal processing and machine learning techniques.[6] This stage is critical because the massive volume of data generated by billions of devices is often redundant, noisy, or incomplete, requiring advanced "truth-finding" algorithms to resolve conflicts among multisource data and ensure the reliability of the system's state estimation.

Beyond basic aggregation, modern IoT analytics focuses on knowledge discovery and generation, where the system identifies patterns and predicts future behaviors rather than merely reporting historical data. This involves complex procedures such as **time-series data cleaning**, which transitions from simple anomaly detection to sophisticated anomaly repairing to maintain the continuity of the digital thread.

Furthermore, optimized sensor configuration and feature selection are employed for **advanced fault detection and diagnosis**, ensuring that the most relevant metrics are prioritized for processing. These analytical capabilities allow the system to handle the emergent behaviors of complex environments, effectively bridging the gap between raw physical sensing and the high-level intelligence required for autonomous control and decision-making.

2.2 Digital Twin

A Digital Twin (DT) represents a dynamic digital replica of physical assets, processes, and systems that comprehensively monitors their entire life cycle . As systems transition from being merely "complicated" to "complex", so characterized by numerous components, multiple interface points, and emergent behavior, DT technology has become the core framework for organizing and managing system information.[9]

2.2.1 Digital twin model and lifecycle categories

The conceptual ideal of a Digital Twin consists of three main elements: a physical element in "real space" a virtual representation in "virtual space" and the Digital Thread that facilitates the bidirectional flow of data and information between them. This model is applied across three distinct lifecycle categories:

- **Digital Twin Prototype (DTP)**: Resides in the "Create" phase; it contains the design specifications, requirements, and simulations (for example CAD models) needed to build a physical product
- **Digital Twin Instance (DTI)**: Corresponds to an individual, unique physical product ("Physical Twin"); it captures the specific as-built data and real-time operational history of that single unit
- **Digital Twin Aggregate (DTA)**: An aggregation of all DTIs for a product population, enabling the correlation of input data with performance outputs to assess and predict system-wide behaviors.

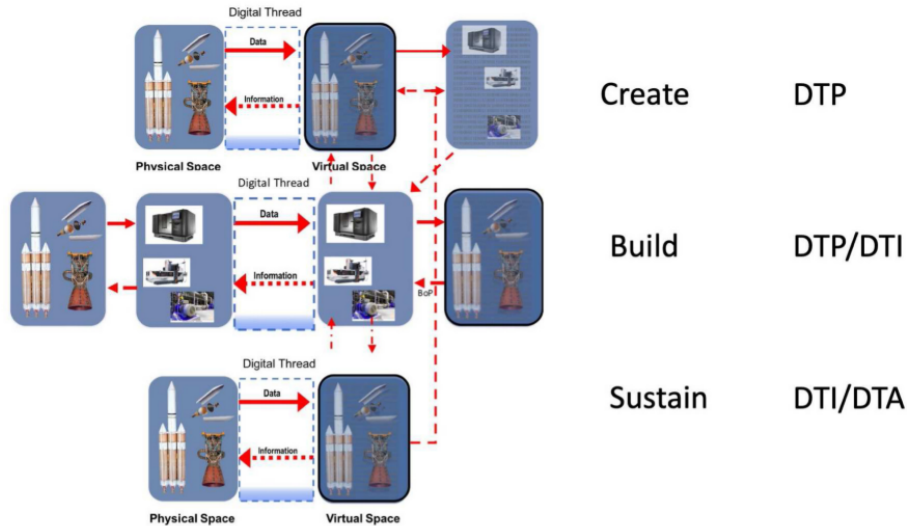


Figure 2.2: Digital twin lifecycle[9]

2.2.2 Symbiotic Connection to IoT

IoT serves as the foundational "backbone technology" for the Digital Twin, enabling the real-time, multisource data collection required for high-fidelity synchronization[6][9].

The "reflectivity" of a digital twin (its ability to accurately mirror its physical counterpart) relies entirely on the sensor modalities provided by IoT networks, such as temperature, vibration, and rotation. By providing a continuous data thread, IoT allows the virtual twin to perform "Tests of Virtuality," including behavior tests to ensure a system meets design requirements and external force tests to simulate stress scenarios in a virtual environment before real-world execution [9].

2.2.3 Intelligent Digital Twins

As systems evolve toward higher complexity, traditional digital twins are transitioning into Intelligent Digital Twins (IDTs), which incorporate artificial intelligence to manage emergent behaviors[9][10]. Unlike passive models, IDTs are defined by four primary attributes:

- **Active:** The IDT is not a passive data store; it initiates tasks and actively provides information through "cued availability": delivering the right data and processes to human operators exactly when needed to augment their decision-making.
- **Online:** The twin remains continuously connected to its physical counterpart via the Digital Thread. This online connectivity allows for real-time state information to flow from the physical twin into the virtual model, ensuring the digital replica reflects the current operational reality.
- **Goal seeking:** Intelligence in the twin is driven by the intent to continually match actual performance against defined requirements. The IDT learns as the product matures, allowing it to compensate for physical degradation and make adjustments necessary to maintain required performance levels.
- **Anticipatory:** Levering Front Running Simulation (FRS), the IDT is constantly running simulations of the product as it performs its duties. This capability allows the twin to move forward in time to a future state to predict performance outcomes and potential failures before they manifest in the physical world.

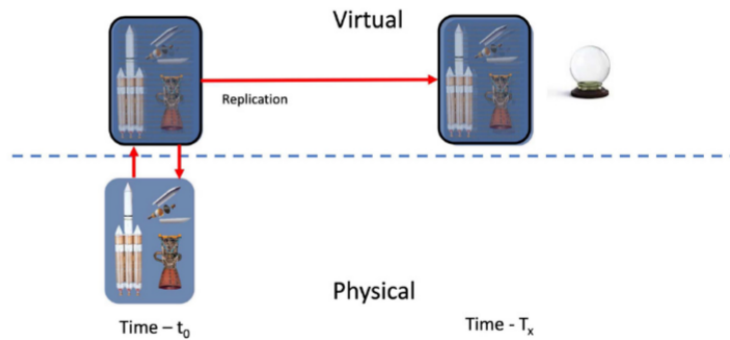


Figure 2.3: Front Running Simulation (FRS)[9]

By integrating these attributes, Intelligent Digital Twins serve to augment human intelligence, processing vast and complex datasets in an accurate, unbiased manner to support the development and management of increasingly sophisticated IoT systems.

2.3 SAIBA framework

The SAIBA framework (representing Situation, Agent, Intent, Behavior, and Animation) is an international standard designed to unify the generation of behaviors in embodied agents [5]. It proposes a modular, three-stage model that abstracts the "intent" of an agent from its physical "realization," providing a structured approach to building sophisticated autonomous systems [11].

2.3.1 Stages of the SAIBA architecture

The SAIBA framework is defined by three distinct processing stages[5] that manage the transition from high-level reasoning to physical action:

- **Intent Planning:** This first stage determines the goals, beliefs, and intentions of the agent based on its current situation. The output is a functional description of the desired state, expressed through the Function Markup Language (FML), which describes what the system should achieve without specifying how it should physically behave.
- **Behavior Planning:** This stage takes the functional intent (FML) and translates it into a detailed plan of physical actions. This plan is represented using the Behavior Markup Language (BML), a standard XML-based language that describes the multimodal physical realizations required to fulfill the intent.
- **Behavior Realization:** The final stage involves the actual execution of the planned behaviors through a realization engine, which drives the physical or virtual components of the system.

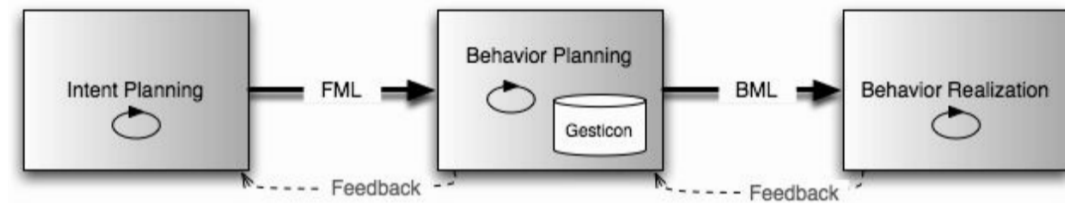


Figure 2.4: SAIBA architecture[5]

2.3.2 Integration of reactive behaviour

While the standard SAIBA model focuses on deliberate planning, the framework can be extended to include a reactive layer. Reactive behaviors are short-latency responses to environmental stimuli (such as a startle response) that must bypass high-level cognitive planning to ensure system safety and responsiveness [11]. This reactive process can directly influence the behavior planning or realization stages, allowing the system to handle unexpected events or sensor-triggered alarms in real-time.

2.3.3 Application of the framework

The modularity inherent in the SAIBA framework provides a robust methodological foundation for supporting the design and deployment of IoT scenarios through Agentic AI. In this paradigm, the Intent Planning stage serves as the primary Agentic AI

layer, where autonomous agents reason over multisource IoT data to determine high-level operational goals. These goals are expressed through FML in a hardware-agnostic manner, allowing designers to define complex scenarios based on functional requirements (such as optimizing energy consumption with specific constraints) without being constrained by the low-level protocols of specific IoT devices[5][11].

This separation of concerns significantly enhances deployment flexibility, as the Behavior Planning stage acts as a translator that maps general functional intents into site-specific BML actuation plans tailored for the heterogeneous "things" within a specific network. Furthermore, the integration of a Reactive Behavior module ensures the operational resilience of the IoT scenario by enabling immediate, low-latency responses to critical sensor alerts at the edge. By bypassing the slower deliberate reasoning of the central Agentic AI for emergency actions, the system maintains high-speed responsiveness and safety, effectively bridging the gap between autonomous "mind" and distributed "body" in sophisticated IoT environments.

2.4 Agentic AI

Agentic AI represents an evolutionary leap in artificial intelligence, moving beyond systems that merely generate content to those that possess autonomous agency [12]. It is defined as an AI system capable of goal-directed behavior, dynamic adaptation, and self-improvement with minimal to no human intervention [13]. Unlike traditional AI, which is primarily reactive, Agentic AI is characterized by proactivity, the ability to anticipate needs, initiate tasks, and act independently to solve complex, multi-step problems [12] [13].

A core defining feature of this paradigm is its "chaining" capability: the ability to sequence multiple actions autonomously in response to a single high-level request. It functions not as a tool, but as an intelligent partner that perceives its environment, reasons through sub-goals, and iteratively refines its strategies to achieve a defined objective [13][14].

2.4.1 Conceptual taxonomy and differences

To understand Agentic AI, it must be distinguished from its precursors and related paradigms. The evolution of artificial intelligence has transitioned from passive, content-generating systems toward highly autonomous, orchestrated ecosystems. Understanding the distinctions between Generative AI, AI agents, and Agentic AI requires an analysis of their varying levels of autonomy, architectural complexity, and operational intent.

Autonomy

The primary differentiator in autonomy is the shift from human-triggered outputs to self-initiated actions[14]. Generative AI represents the lowest level of autonomy, as it is strictly reactive and prompt-dependent; it produces content only when explicitly requested by a user[12]. AI Agents possess bounded autonomy, meaning they can perform specific tasks independently once initialized, such as querying a database or executing a tool-based workflow, but they remain confined to a narrow domain. Agentic AI signifies the frontier of autonomy, characterized by "proactivity": the ability to anticipate

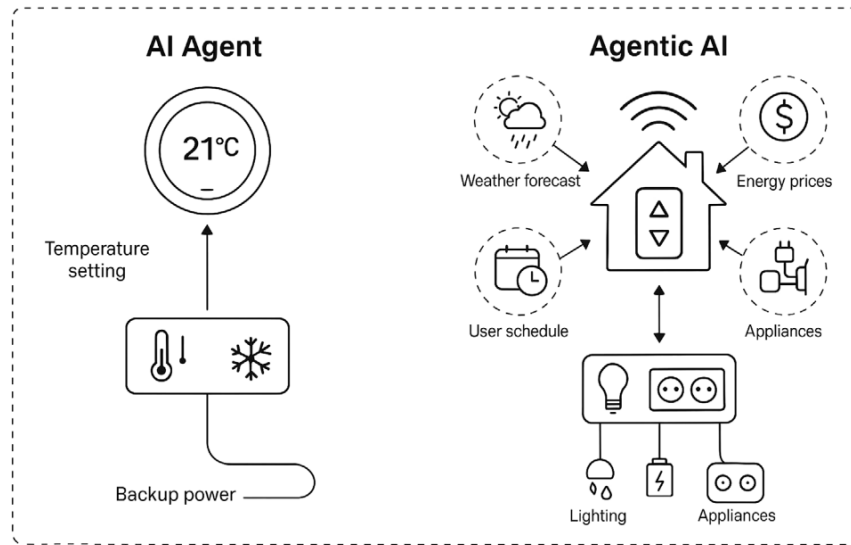


Figure 2.5: Comparative illustration of AI Agent versus Agentic AI

human problems, initiate tasks independently, and operate in dynamic environments with minimal to zero human intervention.

Architectural Complexity

Architectural progression marks the move from monolithic structures to distributed intelligence[12]. Generative AI typically utilizes a single, stateless model optimized for content synthesis. AI Agents increase complexity by augmenting the core model with external tool integration, APIs, and sequential reasoning loops. Agentic AI employs the most complex architecture: a multi-agent ecosystem. These systems utilize specialized agents coordinated by a "meta-agent" or orchestrator, supported by persistent memory architectures (episodic, semantic, and vector) and shared context layers to maintain continuity across long-term objectives[14].

Operational Intent

The intent of these systems has moved from creating information to executing workflows. Generative AI is designed for the synthesis of multi-modal content, such as text or code. AI Agents are task-specific, intended to automate well-defined functions like customer support triaging or internal information retrieval[12]. Their intent is to solve narrow problems by applying specific tools to specific data[13]. Agentic AI focuses on high-level orchestration, designed to handle complex, multi-faceted goals by autonomously decomposing them into sub-tasks and managing the entire problem-solving journey from start to finish.

Feature	Generative AI	AI Agent	Agentic AI
Autonomy	Reactive; prompt-driven	Bounded; goal-directed	Proactive; autonomous
Architecture	Single model; stateless	Model + Tool integration	Multi-agent ecosystem
Intent	Content synthesis	Task automation	Strategic orchestration
Memory	No/Short context window	Task-specific cache	Persistent and shared
Logic	Pattern selection	Rule-based/Supervised	Meta-learning/RL

Table 2.1: Taxonomy of AI Paradigms

2.4.2 Core components

The architecture of an Agentic AI system is composed of several interconnected functional layers that facilitate its autonomous operation:

- **Perception Layer:** Utilizes sensors and digital interfaces to collect data (text, images, or real-world signals) from the environment [14]. In modern systems, Large Image Models (LIMs) and multi-modal LLMs serve as the primary perceptual engines [12]
- **Cognitive Reasoning and Planning:** The "brain" of the agent, typically powered by an LLM, which performs task decomposition, evaluates potential strategies, and selects the most optimal course of action [12] [14]
- **Action Execution and Actuation:** Translates cognitive decisions into real-world interactions through an action library, often involving external tool use via APIs [12]
- **Memory Architectures**[12]:
 - **Episodic Memory:** Records past interactions and short-term task history
 - **Semantic Memory:** Stores long-term facts and structured domain knowledge
 - **Vector Memory:** Enables similarity-based retrieval (RAG) to ground responses in factual data
- **Reflection and Evaluation:** Verify immediate or final outputs, critique candidates plans, and trigger selective replanning.
- **Orchestration:** Interaction modules supports human-agent and agent-agent dialogue (clarification, negotiation ...) and surface trace information (actions, tools, sources...) for transparency. Autonomy emerges when perception, planning execution, memory and reflection are orchestrated over time; in many systems, an LLM-based supervisor coordinates sub-agents, invokes memory or tools, and maintains coherence across steps.

2.4.3 Types of architectures

Architectures for Agentic AI have evolved from simple rule-based logic to complex, distributed systems. In this section there are an explanation of the principal types with the relative structure based on the components already explained in the last section.

ReAct Single-Agent

The ReAct (Reasoning and Acting) architecture is an iterative framework designed to overcome the limitations of single-pass inference. It synergizes reasoning traces with task-specific actions to allow an agent to process information more deliberately.

The agent operates in a loop where it first generates a "thought" (reasoning trace), executes an "action" (calling a tool or API), and then "observes" the result from the environment before updating its next thought.

This loop allows the agent to verify retrieved data before summarizing it, which significantly reduces hallucinations and improves logical consistency in multi-step workflows. It effectively bridges the gap between internal reasoning (Chain-of-Thought) and external environment interaction.

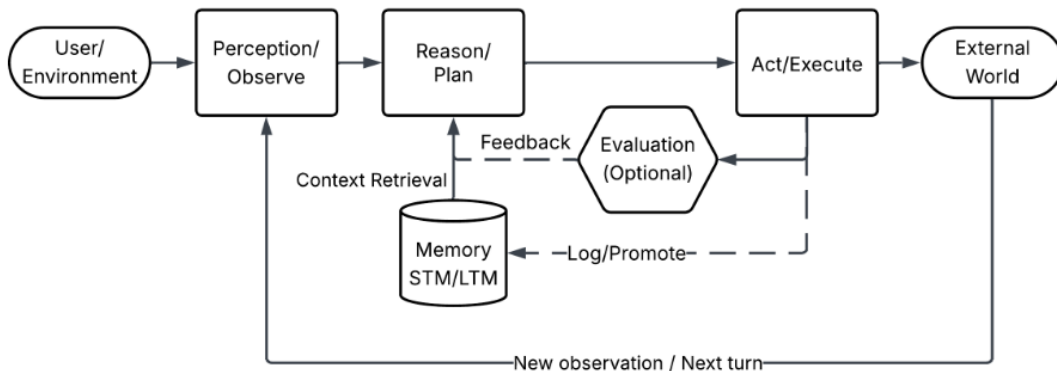


Figure 2.6: ReAct Single-Agent architecture[14]

Supervisor/Hierarchical

The Supervisor or Hierarchical architecture moves beyond isolated execution to a managed multi-agent ecosystem. It introduces a meta-agent that coordinates the behavior of subordinate specialized entities.

In the specific a central supervisor parses a high-level goal into smaller, manageable sub-tasks and distributes them across a network of agents. The supervisor manages task dependencies, assigns roles to agents based on their specialization (e.g., a "Coder" agent or a "Reviewer" agent), and resolves conflicts between agent outputs.

Hybrid reactive-deliberative

This architecture combines the speed of rule-based systems with the strategic depth of planning-based systems. It runs two coordinates loops in parallel.

A fast reactive loop that handles time-critical events and a slower deliberative loop that maintains goals and plans. The loops share a common memory and are overseen by an Arbitrator that resolves conflict when immediate reactions and long-horizon plans disagree.

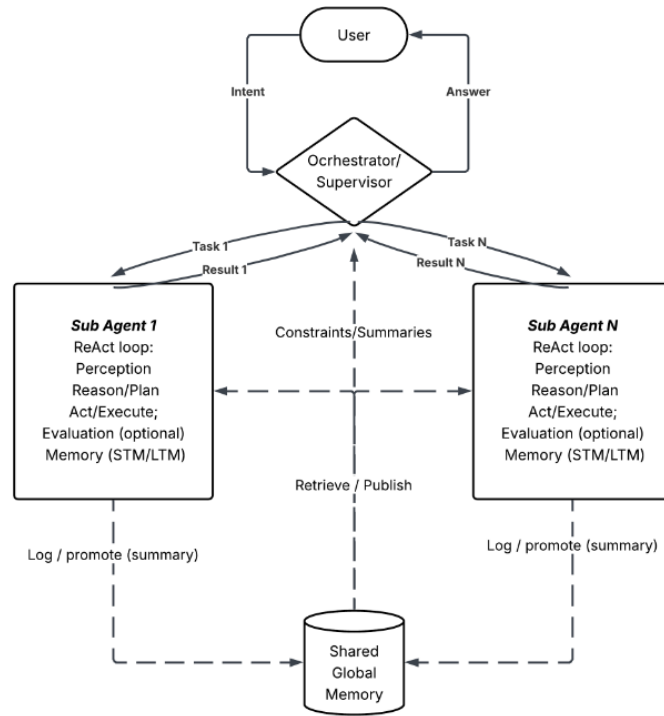


Figure 2.7: Supervisor/hierarchical architecture[14]

Belief-Desire-Intention

BDI is a classical cognitive architecture inspired by human practical reasoning and is foundational to modern agent theory. In this architecture there are several states: beliefs(worlds state), desires(candidates goals), and intentions(the subset of the goals the agents want to pursue). After intentions there is the real act and the cycle will repeat again.

Layered Decision (Neuro-Symbolic)

Layered decision architectures integrate neural perception and probabilistic inference with a symbolic planner and an execution layer, typically followed by evaluation and memory update. The aim is to handle uncertainty via statistical inference while preserving interpretability and traceable decisions through symbolic planning.

2.5 Agents interoperability protocols

The advancement of Multi-Agent Systems (MAS) has transitioned from isolated Large Language Models (LLMs) to complex ecosystems where autonomous agents must interact with diverse tools, data sources, and other agents. Traditionally, these interactions were managed through "glue-code": custom, brittle integrations that are difficult to scale and secure [15]. To move toward a more scalable "Agent Economy," several standardized protocols have been proposed to handle different tiers of interaction: vertical (agent-to-infrastructure) and horizontal (agent-to-agent) [15] [16].

2.5.1 Model Context Protocol (MCP)

The Model Context Protocol (MCP), introduced by Anthropic, is a vertical integration standard designed to provide a universal interface between AI models and data sources [15]. Often described as the "USB-C for AI," MCP replaces the need for bespoke connectors by using a model-agnostic JSON-RPC client-server architecture [17][16].

This protocol allows applications to securely expose "resources" (static or dynamic data), "tools" (executable functions), and "prompts" (predefined templates) to an LLM [17]. By standardizing these interfaces, MCP enables agents to interact with complex environments, such as IoT data spaces and industrial context brokers, without requiring the developer to write custom code for every new integration.

Architectural Model

MCP employs a Client-Server architecture where the components are decoupled to ensure security and modularity[15]:

- **MCP Host:** The primary application (an IDE, a desktop assistant, or a custom LLM app...) that orchestrates the overall workflow and manages the user's session
- **MCP Client:** A component within the host that handles the protocol-specific logic, establishing and maintaining connections to one or more servers
- **MCP Server:** A lightweight provider that exposes specific datasets or functionalities to the model. A single host can connect to multiple servers simultaneously to aggregate different capabilities

Core Components

The protocol exposes three core primitives that define how the LLM interacts with the server:

- **Resources:** Read-only data sources, such as file contents, database records, or API logs, which are provided as context to the model
- **Tools:** Executable functions that allow the model to take actions in the external world. These are defined with clear schemas (for example JSON Schema) so the model knows how to parameterize each call
- **Prompts:** Predefined interaction templates or "shortcuts" provided by the server to help the model perform specific tasks effectively
- **Sampling:** A feature that allows the server to request an LLM completion back from the host, enabling the server to leverage the model's reasoning for its own internal logic

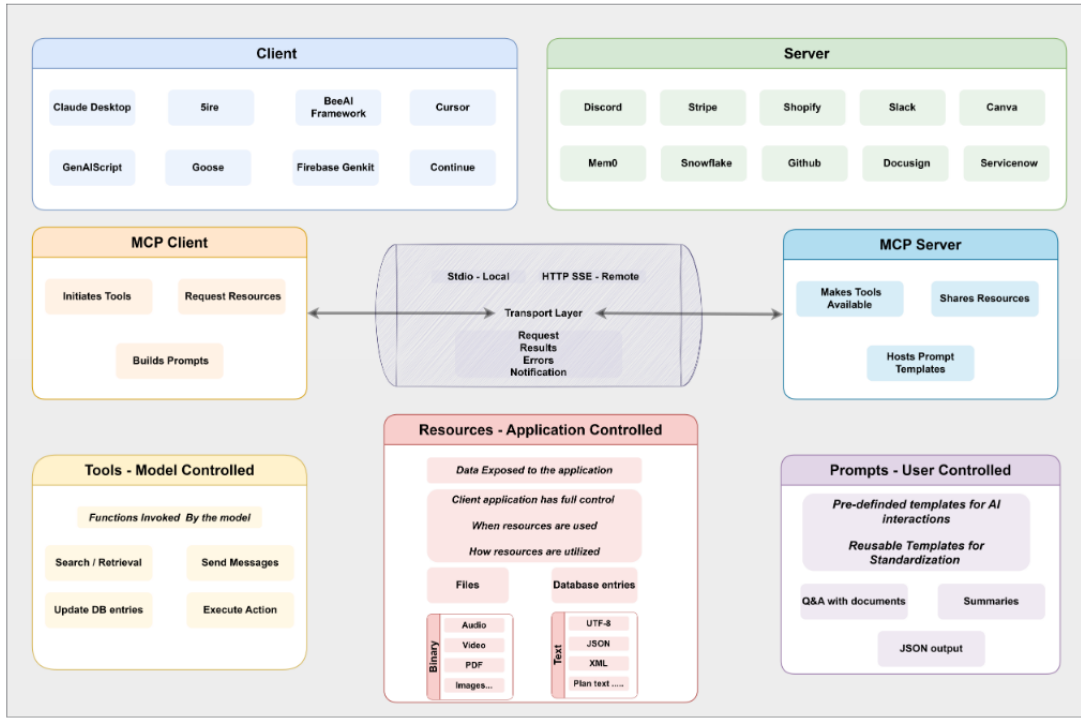


Figure 2.8: MCP architecture[16]

2.5.2 Agent To Agent(A2A)

While MCP handles how an agent accesses tools, Google’s Agent-to-Agent (A2A) protocol focuses on horizontal communication, enabling agents from different vendors and frameworks to collaborate [16]. The A2A framework utilizes ”Agent Cards”(standardized descriptions of an agent’s capabilities and security requirements)to facilitate discovery and negotiation between autonomous entities. This protocol primarily utilizes HTTP and Server-Sent Events (SSE) to manage asynchronous task delegation and multimodal information exchange [15].

Architectural model

A2A utilizes a Peer-like relationship between two distinct agent roles:

- **Client Agent:** The agent that represents the user’s initial intent. It is responsible for planning the workflow and delegating specialized sub-tasks to other agents
- **Remote Agent:** A specialist agent that receives task delegations. It exposes its capabilities through standardized discovery mechanisms and executes the work on behalf of the Client Agent

Core components

The protocol is built around several key structural elements:

- **Agent Card:** A structured JSON metadata document that serves as the agent’s ”business card.” it declares the agent’s identity, supported skills, input/output formats, and authentication requirements

- **Tasks:** The fundamental unit of work in A2A. A task encapsulates the request parameters and has a unique lifecycle, moving from initiation to execution and finally to completion
- **Artifacts:** The formal outputs produced by the Remote Agent. Artifacts can include structured data, text, or file references that are returned to the Client Agent
- **Communication Layer:** A2A utilizes HTTP POST for initiating requests and Server-Sent Events (SSE) for streaming progress updates and intermediate status reports, enabling long-running, asynchronous collaborations

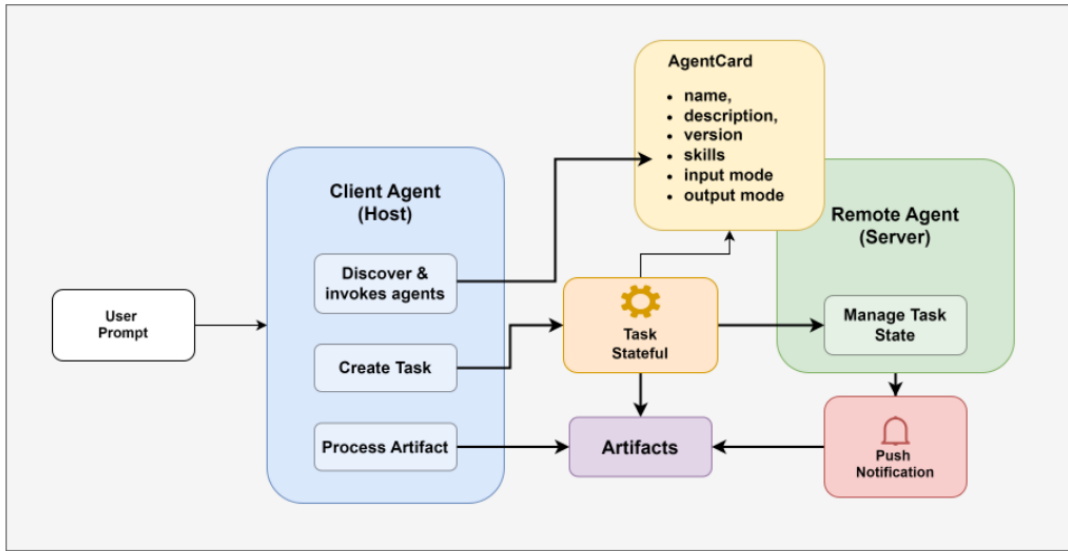


Figure 2.9: A2A architecture[16]

2.5.3 Agent Communication Protocol (ACP)

The Agent Communication Protocol (ACP) architecture is a modular, HTTP-based system designed to facilitate interaction between clients and agents. It is composed of three primary components: the ACP Client, the ACP Server, and one or more ACP Agents[16].

The **ACP Client** initiates communication by submitting requests in ACP-compliant format.

The **ACP Server** acts as a middleware component, translating external HTTP requests into internal agent executions. It manages agent lifecycle phases including execution and optional session persistence.

The **ACP Agent** is the execution component defined using a decorator-based configuration.

Central to this architecture is the **Agent Detail**, a self-descriptive JSON or YAML document that serves as the agent's public identity and capability profile. Complementing this, **Discovery Mechanisms** allow clients to locate agents dynamically at runtime. Once an agent is located, clients issue a **Task Request**, a structured unit of delegated work. All requests and responses conform to ACP's **Message Structure**,

which standardizes the communication envelope. Finally, the result of agent execution is encapsulated in one or more **Artifacts**.

2.5.4 Agent Network Protocol (ANP)

The Agent Network Protocol (ANP) represents a decentralized approach to agent interoperability, focused on creating an open, global "Agent Economy" [15]. Unlike the client-server models of MCP or ACP, ANP adopts a pure peer-to-peer (P2P) architecture that eliminates the need for centralized intermediaries or platform-specific gateways [16].

The protocol relies on W3C Decentralized Identifiers (DIDs) for secure, self-sovereign authentication and JSON-LD graphs for discovery. This allows agents to autonomously find and negotiate services with one another over the open internet, using verifiable credentials to establish trust dynamically. ANP is designed for scenarios where agents from disparate organizations must collaborate without prior manual configuration or a shared governing entity.

2.5.5 Structural patterns and combinations

The effectiveness of these protocols is maximized when they are applied in specific structural combinations, creating a tiered hierarchy of agent capabilities[15]

MCP and A2A synergy

The most common structural pattern is the A2A-MCP Hybrid, which separates high-level orchestration from low-level execution:

- **Orchestration Layer (Horizontal/A2A):** A Client Agent (the Manager) uses A2A to discover a Remote Agent (the Specialist). The Manager delegates a high-level task through an A2A request.
- **Execution Layer (Vertical/MCP):** The Specialist Agent, upon receiving the task, acts as an MCP Host. It invokes its internal MCP Servers to access the specific local data or tools required to solve the sub-task.
- **Result Aggregation:** The results from the MCP tool calls are synthesized by the Specialist and returned to the Manager as structured A2A Artifacts

This pattern can be implemented in 3 different ways:

1. **A2A Agent Utilizing MCP Internally (Primary Pattern):** An A2A server agent uses MCP internally.
2. **Exposing MCP Tools via A2A Agent Card:** A2A skills directly represent MCP tools.
3. **A2A for Tool Orchestration (Alternative/Edge Case):** Using A2A directly for complex "tools".

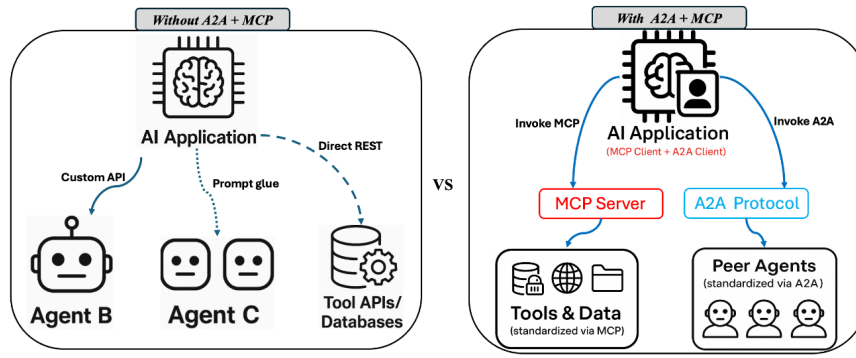


Figure 2.10: MCP+A2A integration[15]

Protocol Evolution and Maturity Pattern

Knowing all the protocols it's important to question their application in their proper context. Research suggests a phased structural roadmap for organizational adoption, moving from internal tool standardization to external network participation[16]:

- **Phase I (Resource Access):** Implementation of MCP to standardize how internal models interact with legacy databases and proprietary APIs.
- **Phase II (Service Interaction):** Adoption of ACP to enable structured, multimodal communication between different internal agent services.
- **Phase III (Inter-Agent Collaboration):** Integration of A2A to allow agents to discover one another and delegate tasks across departmental or organizational boundaries.
- **Phase IV (Global Economy):** Transition to ANP to permit these agents to participate in decentralized, internet-scale marketplaces.

2.6 Agentic AI frameworks

While interoperability protocols define the external communication standards and boundaries between independent systems, Agentic AI frameworks provide the internal execution runtimes, programmatic abstractions, and state management mechanisms required to construct autonomous entities. In an enterprise-grade or distributed multi-agent deployment, these frameworks implement the internal cognitive loops and application logic of an agent, which can then be exposed to external networks through protocols such as MCP, ACP, or A2A.

2.6.1 LangGraph

LangGraph is a specialized orchestration framework[18] within the LangChain ecosystem engineered specifically for constructing stateful, multi-agent runtimes capable of handling cyclic execution paths. Unlike linear piping pipelines, LangGraph models agent behaviors explicitly as state machines represented by directed graphs.

The control flow of a LangGraph application is governed by two structural components: **Nodes** and **Edges**. **Nodes** represent discrete computational units, such as a

model reasoning step, a specific tool execution, or a sub-agent call. **Edges** define the routing logic between these nodes, utilizing either deterministic pathways or conditional functions that analyze the current application state to determine the next execution step.

State management within LangGraph relies on a centralized state object that is passed sequentially between nodes, with each node appending or modifying the shared context. This framework provides granular control over memory persistence layers and transactional checkpointing, making it highly suitable for complex multi-agent setups that require transactional state rollbacks, strict path boundaries, and human-in-the-loop approvals. It integrates smoothly with vertical protocol layers like MCP by mapping graph execution nodes directly to remote MCP tool servers[15].

2.6.2 LangChain

LangChain is a foundational, open-source orchestration library[19] engineered to simplify the development of applications powered by Large Language Models (LLMs) by abstracting common integration patterns into reusable components. Rather than forcing developers to write custom wrappers for model APIs and prompt formatting, LangChain provides a standardized interface for interacting with foundational AI primitives.

The framework structures application logic around several fundamental building blocks, including unified model wrappers, prompt templates, and structured parsers. The defining characteristic of the library is the concept of a "chain" which allows developers to link separate components together sequentially, automatically feeding the output of one step as the input to the next.

Furthermore, LangChain features extensive built-in utility interfaces for loading document retrievers, vector indices, and custom external toolkits. While it serves as the underlying utility layer for many advanced graph runtimes, its core linear piping mechanics are optimized for data pre-processing, sequential extraction pipelines, and simple single-agent automation tasks.

2.6.3 LLamaIndex

LlamaIndex is a data-centric orchestration framework[20] focused primarily on connecting LLMs with private, heterogeneous data spaces. It serves as a data ingestion, indexing, and retrieval middleware layer designed to power robust Retrieval-Augmented Generation (RAG) and data-intensive agent workflows.

The internal architecture of LlamaIndex is divided into three core subsystems: **data connectors**, **index structures**, and **query engines**. **Data connectors** ingest raw data formats from disparate external sources (such as local PDFs, cloud APIs, and relational databases); and then parse them into standard document objects. The **indexing system** then transforms these documents into optimized semantic structures, including vector stores, chunked hierarchical keyword trees, or knowledge graphs.

For agentic operations, LlamaIndex uses conversational **query routing** where the model dynamically selects the optimal retrieval index or storage endpoint based on user intent. This deep integration with data layer semantics allows agents to execute precise multi-step data lookups and reasoning over enterprise silos, aligning structurally with the resource-handling components of the Model Context Protocol (MCP).

2.6.4 Microsoft Agent Framework

Moving beyond its earlier frameworks like Semantic Kernel and Microsoft AutoGen, Microsoft consolidated its enterprise orchestration strategy into the unified Microsoft Agent Framework[21]. This framework is an enterprise-grade orchestration layer designed to manage complex, stateful agent topologies natively within managed cloud environments.

The framework abstracts away manual memory handling by relying on declarative agent configurations and managed, session-aware state stores. It shifts the development paradigm toward defining scalable "Agent Services" that can securely invoke native cloud functions and API plugins. For large-scale system integration, it provides native bindings to streaming data services and IoT platforms, allowing agents to ingest real-time telemetry securely. Because it is optimized for heavy backend integration and strict enterprise compliance, it serves as an ideal deployment platform for RESTful network layers like ACP, while its local plugin architecture maps cleanly to MCP tool execution.

2.6.5 CrewAI

CrewAI is a lean, lightning-fast Python framework[22] built entirely from scratch, completely independent of LangChain or other agent frameworks. It empowers developers with both high-level simplicity and precise low-level control, ideal for creating autonomous AI agents tailored to any scenario. It provides a high-level, role-based orchestration layer designed to mirror traditional human workplace organizational structures and management strategies. Instead of exposing low-level code parameters directly, the framework encapsulates language models within discrete autonomous personas characterized by explicit roles, specific goals, and detailed backstories that shape their reasoning boundaries.

These specialized agents are grouped into a modular container called a **Crew**, which orchestrates how tasks are executed across the collective. The framework supports multiple operational processes(**Flows**), such as sequential execution paths where tasks are passed down a line, or hierarchical management chains where a supervisor agent reviews outputs and delegates work dynamically. By prioritizing role-playing and persona-based guardrails, CrewAI is frequently used to automate complex business operations. This task-centric delegation design maps cleanly onto the Client-Remote agent delegation flows found in horizontal multi-agent networks.

2.6.6 OpenAI Agents SDK

The OpenAI Agents SDK is an official, open-source Python framework[23] designed to provide developers with a production-ready toolkit for building multi-agent workflows. Evolving beyond early experimental frameworks, it simplifies complex orchestrations into a lightweight, provider-agnostic loop capable of supporting diverse LLM backends.

The architecture relies on a core set of fundamental primitives: **Agents** (which encapsulate model instructions and tools), **Handoffs** (a specialized tool call mechanism for transferring execution control and context between agents), **Guardrails** (configurable safety checks for input/output validation), and **Sessions** (automated conversation history management). By stripping away overly complex state abstractions in favor of a robust runner loop that automates tool execution and message history, the SDK offers

a highly predictable and trackable execution model. This clean separation of agent instructions, handoffs, and tool schemas maps perfectly to the structured JSON-RPC execution loops found in vertical connection protocols like MCP.

2.6.7 Comparative evaluation

Selecting an orchestration runtime requires analyzing how its internal state handling and control flow mechanics interface with external network standards. The table below provides a comparative evaluation across key engineering dimensions, highlighting how their internal designs dictate their affinity for specific interoperability protocols.

Framework	Core Software Paradigm	State Management Model	Control Flow Determinism	Optimal Protocol Affinity
LangGraph	Stateful, Cyclic Directed Graphs	Built-in state persistence with short/long-term memory	High (Explicitly defined by graph edges and nodes)	MCP (Binds tool servers to graph nodes)
LangChain	Linear Abstraction Chaining	Stateless pipelines with optional external memory attachments	High (Follows fixed, predetermined chain sequences)	MCP (Maps tool wrappers to model utilities)
LlamaIndex	Data-Centric Indexing & Retrieval	Index-bound context tracking and document node stores	Medium (Semantic data routing based on query intent)	MCP (Aligns with vertical resource handling primitives)
Microsoft Agent Framework	Managed Enterprise Agent Services	Declarative, session-aware managed cloud stores	High (Strict routing tied to cloud infrastructure)	ACP / MCP (Aligns with heavy web services and local plugins)
CrewAI	Role-Based Crew Operations	Task-centric context propagation across group members	Medium (Structured by sequential/hierarchical rules)	A2A (Fits Client-Remote task delegation flows)
OpenAI Agents SDK	Provider-Agnostic Primitives & Handoffs	Automated session management and trace logs	Medium (Governed by functional execution handoffs and guardrails)	MCP (Mirrors basic lightweight RPC primitives)

Table 2.2: Comparative Evaluation of Frameworks

3. Literature Review

The rapid expansion of the Internet of Things (IoT) has introduced substantial operational complexity, and while Intelligent Digital Twins offer profound theoretical capabilities, the orchestration and configuration of these heterogeneous infrastructures remain a significant bottleneck. Consequently, bridging the semantic gap between an operator's abstract, high-level intent and the low-level actuations required across diverse platforms is a primary challenge in modern IoT ecosystems.

To systematically evaluate the state-of-the-art solutions attempting to address this gap, this literature review analyzes existing frameworks and approaches. The goal of this review is to identify the current capabilities and limitations of existing systems, highlighting the necessity for a novel agentic architecture. To ensure a rigorous and structured comparison, the reviewed works are evaluated against a predefined matrix of characteristics. These characteristics represent the core functional requirements necessary to achieve the objectives of this thesis (empowering IoT architects to deploy complex infrastructures autonomously using Agentic AI).

The comparative evaluation is grounded in the following characteristics:

- **C1 Natural Language Intent Support:** Does the system accept abstract, unstructured natural language as the primary method to initiate the deployment process?
- **C2 Hierarchical Multi-Agent Architecture:** Does the system distribute deployment tasks across a hierarchical network of specialized, context-sharing entities rather than a centralized execution?
- **C3 Automated IoT Deployment:** Does the system automatically execute the physical deployment of IoT configurations and infrastructure, eliminating the need for manual, device-by-device provisioning by human operators?
- **C4 Dynamic Workflow Resilience:** When encountering a disruption or dynamic change during execution, does the orchestration engine retain its workflow state and dynamically adapt its path, rather than failing and requiring a complete system restart?
- **C5 Context and Memory Tracking:** Does the system implement persistent context and memory tracking across multi-step workflows, allowing it to remember past actions, delegated sub-tasks, and environmental history without losing the thread of execution?
- **C6 AI-Driven Reasoning Capability:** Does the system employ an AI-driven reasoning engine to autonomously deduce and generate the sequence of deployment actions, rather than relying on static, pre-programmed rules, templates, or deterministic scripts?

3.1 Data collection methodologies

In order to collect papers relevant for the literature review, the methodology used is to exploit a search query in Scopus. The definition of a search query is based on the so-called keywords, elementary terms that, joined together, allow extracting the desired results: they are joined by means of logical operators (such as AND, OR, NOT), so as to define the way in which the same terms will be searched:

("Agentic AI" OR "LLM Agent*" OR "Autonomous Agent*" OR "Multi-Agent" OR "MAS") AND ("Internet of Things Deployment" OR "IoT Deployment") AND ("Architecture" OR "Framework" OR "Design")

3.1.1 Inclusion and exclusion criteria

To evaluate the state of the art in Intelligent IoT and Agentic architectures, a systematic selection process was applied to the collected bibliography.

The inclusion of research was governed by three primary criteria:

- architectural relevance to multi-agent orchestration
- alignment with established system characteristics (mentioned previously in the introduction of the literature review)
- methodological diversity

Papers were prioritized if they presented alternative orchestration strategies (such as Hierarchical Deep Multi-Agent Reinforcement Learning) which serve as essential baselines for contrasting traditional statistical optimization against generative, intent-based agentic workflows.

Conversely, papers were excluded if they focused exclusively on low-level physical layer metrics (for example raw packet scheduling or specific hardware-level energy consumption) without addressing higher-level semantic reasoning or system-wide orchestration. Furthermore, studies that repeated existing, well-documented methodologies already established in the background chapter were omitted to ensure the literature review maintains a focus on innovative approaches to agentic IoT deployment.

3.2 Discussion

3.2.1 Natural Language Intent and Architectural Hierarchy

A significant gap exists across the current literature regarding **natural language intent (C1)**, as none of the analyzed systems natively accept abstract, unstructured text to initiate workflows. Instead, the Hi-MAD framework[25] and the MARL orchestration system [27] rely strictly on structured, numerical metrics like Quality of Service or channel states. While the LLM platform for traffic navigation [26] integrates natural language models, it restricts their use to parsing routing patterns rather than driving broader infrastructure provisioning workflows. Similarly, both the dMCP framework [28] and the softwarized edge architecture[29] process technical system parameters or telemetry inputs directly rather than intuitive human-centric prose.

To manage operational workloads, the **architectural paradigms (C2)** differ significantly. The Hi-MAD framework [25] explicitly implements a strict vertical hierarchy

Table 3.1: Comparative evaluation of literature approaches based on system characteristics.

Approach	C1	C2	C3	C4	C5	C6
X-IoT[24]	✗	✗	✓	✗	✗	✗
Hi-MAD[25]	✗	✓	✓	✓	✓	✓
LLM for traffic navigation[26]	✗	✗	✗	✓	✓	✓
Resources orchestration for IoT[27]	✗	✗	✗	✓	✓	✓
dMCP Framework[28]	✗	✗	✓	✓	✗	✓
Softwarized Edge Intelligence[29]	✗	✗	✓	✓	✗	✗

C1: Natural Language Intent Support

C2: Hierarchical Multi-Agent Architecture

C3: Automated IoT Deployment

C4: Dynamic Workflow Resilience

C5: Context and Memory Tracking

C6: AI-Driven Reasoning Capability

that divides tasks across global, context, and device-level layers. The remaining approaches—including the model-driven X-IoT [24], the traffic navigation model[26], the spectrum management framework[27], the dMCP framework [28], and the data-driven continuum plane[29] operate without this specific hierarchical multi-agent orchestration setup, relying instead on specialized peer nodes, standalone software clients, or direct agent-to-infrastructure communication interfaces.

3.2.2 Deployment Automation and Workflow Resilience

When examining how these systems **automate IoT deployment(C3)**, capabilities vary. Portable model-driven tools like X-IoT[24] and dynamic runtime composition frameworks like Hi-MAD[25] successfully execute automated infrastructure orchestration to eliminate manual device-by-device provisioning. This deployment capability is also shared by the dMCP protocol[28], which dynamically maps intents into executable network policy vectors, and the softwarized edge deployment [29], which utilizes programmable infrastructures and containerized microservices to trigger dynamic adaptations across the cloud/edge continuum. Conversely, the frameworks presented in [27] and [26] focus on edge-layer data processing and physical layer channel coordination, without autonomously deploying systemic infrastructure configurations.

Despite these differences in deployment, **dynamic workflow resilience (C4)** is strongly represented. Except for the static configurations of X-IoT[24], all analyzed frameworks retain workflow execution states under operational drift. The optimization loops in [25], [26], and [27] systematically handle network disturbances and traffic changes. This resilience is replicated in dMCP[28] via its event-driven trigger mechanism that re-evaluates configurations during environmental drops, and within the softwarized architecture[29] through a modular control plane that handles ongoing telemetry variations on-the-fly.

3.2.3 Context Tracking and Autonomous Reasoning

The capability to **maintain persistent context and memory tracking (C5)** remains restricted to specific platforms. The Hi-MAD framework [25], the traffic navigation system [26], and the resource orchestration loop [27] actively track environmental history and historical channel parameters over time. However, the remaining setups—including X-IoT [24], the dMCP framework [28], and the softwarized edge architecture [29], do not preserve persistent context layers or execution histories across sequential multi-step deployment operations.

This stateful gap impacts the corresponding **AI-driven reasoning capabilities (C6)**. Moving away from traditional templates and deterministic scripts, autonomous reasoning is leveraged by Hi-MAD [25] via Deep Reinforcement Learning, by the navigation platform [26] through domain-adaptive language models, by the spectrum manager [27] via cascaded reinforcement loops, and by dMCP [28] through its integrated LLM-driven cognitive engine. On the other hand, the model-driven parameters of X-IoT [24] and the ML-based inference services of the softwarized edge infrastructure [29] (which rely on fixed classification boundaries rather than flexible reasoning algorithms) lack the capacity to independently deduce novel deployment paths.

3.3 Key findings and gaps

3.3.1 Key findings

The systematic evaluation of current literature reveals that dynamic workflow resilience is a well-established strength among modern IoT frameworks. Most state-of-the-art systems successfully retain their execution states and dynamically adapt to operational drift, network disturbances, or environmental changes without requiring a complete manual restart. Furthermore, several frameworks demonstrate robust capabilities in automating physical deployments, effectively eliminating the need for manual, device-by-device provisioning through the use of dynamic runtime compositions and event-driven trigger mechanisms. There is also a notable integration of AI-driven optimization techniques—such as Deep Reinforcement Learning and domain-adaptive models—which are actively employed to handle specific edge-layer data processing and traffic adaptations.

3.3.2 Identified gaps

Despite the strengths observed in existing architectures, several critical limitations prevent current systems from achieving fully autonomous, intent-driven IoT deployment. These gaps can be categorized into the following core areas:

- **The Semantic Communication Barrier:** Existing systems completely lack native support for abstract, unstructured natural language. Current frameworks force operators to initiate workflows using structured numerical metrics, rigid technical parameters, or direct telemetry inputs. This leaves a significant semantic gap between intuitive, human-centric design goals and the low-level execution required by the system.
- **Absence of Unified Hierarchical Orchestration:** The prevailing architectural trend favors specialized peer nodes, standalone clients, or direct software-to-infrastructure interfaces rather than a centralized, multi-agent hierarchy. With

rare exceptions, systems fail to distribute complex deployment tasks across a managed network of specialized, context-sharing entities, significantly limiting their ability to decompose overarching goals into coordinated sub-tasks.

- **Inconsistent Infrastructure Automation:** While some frameworks achieve automated provisioning, a large portion of the literature restricts automation to localized edge-layer data processing or physical layer channel coordination. These systems lack the broader capability to autonomously orchestrate and deploy systemic virtual infrastructure configurations across IoT platforms.
- **State Management and Autonomous Reasoning Deficits:** Many frameworks fail to implement persistent context tracking or retain execution histories across sequential, multi-step deployment operations. This inability to maintain a stateful memory directly hampers advanced reasoning capabilities. Consequently, instead of independently deducing novel deployment strategies for dynamic scenarios, these systems often fall back on deterministic scripts, fixed classification boundaries, or static model-driven templates.

4. Architectural overview

After having a complete view of all the background concepts and having performed a complete literature review on the already existent papers that are concerning IoT deployment, this chapter will address the gaps that have been identified with the design of the solution this thesis want to realize.

4.1 High-level architecture

The proposed architecture introduces a hierarchical, multi-agent orchestration framework designed to address the substantial semantic gap between an operator’s abstract, high-level intent and the low-level, field-specific actuations required for system integration. To mitigate the operational complexity inherent in expanding ecosystems, this framework automates the configuration and instantiation of virtual entities entirely within a centralized IoT platform. In this specific implementation, the physical layer is completely abstracted; the deployment occurs exclusively within a virtualized environment, eliminating direct hardware interaction.

Structurally, the architecture draws foundational inspiration from the Situation, Agent, Intention, Behavior, Animation (SAIBA) framework. By explicitly separating deliberate intent planning from complex behavior realization, the multi-agent ecosystem can process natural language requests, query static Digital Twin configuration models, and independently orchestrate the necessary API parameters on the target virtual platform.

4.2 Interaction and Intent Planning Layer

The initial phase of the deployment loop is governed by the Interaction and Intent Planning layer, which serves as the primary cognitive engine of the system. This layer is responsible for interpreting the operator’s natural language requirements and translating them into actionable, hardware-agnostic functional descriptions.

At the core of this layer operates the **IoT Deployment Orchestrator**. Functioning as the central supervisor meta-agent, the Orchestrator utilizes a Large Language Model (LLM) equipped with Chain-of-Thought (CoT) prompting. When a user submits an unstructured request (for example "I want to deploy 3 movement sensor in the A building, give me a visual that explain me how I can set the configuration"), the Orchestrator autonomously generates a series of intermediate reasoning steps to decompose the complex, multi-step problem into manageable sub-tasks. Instead of producing deterministic, pre-programmed scripts, this agentic reasoning allows the Orchestrator to dynamically map the abstract goals into a structured functional layout, which it then delegates to subordinate specialized agents.

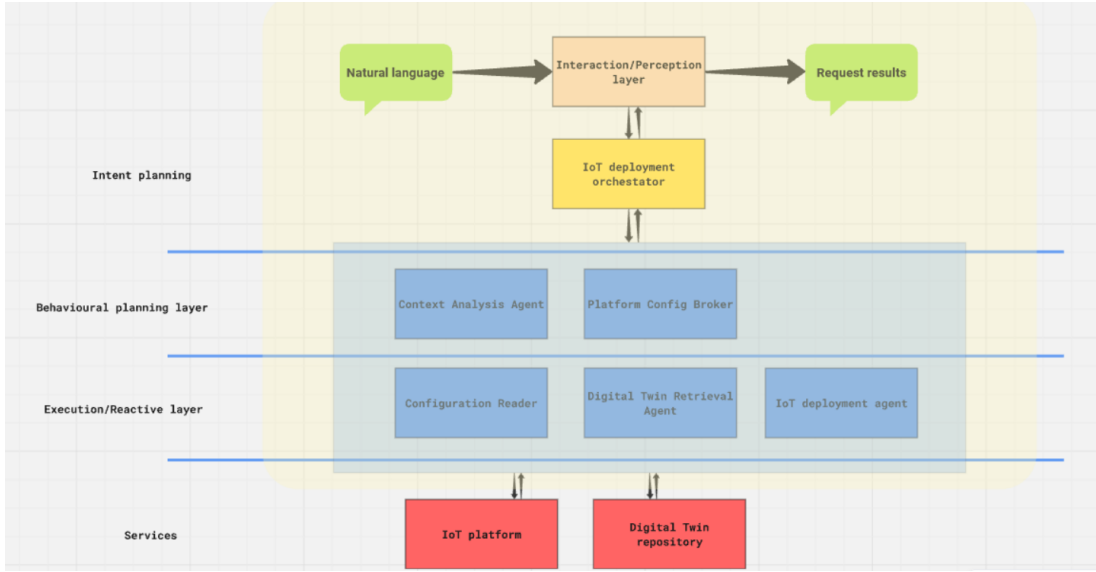


Figure 4.1: System High-level architecture

4.3 Behavior Planning and Specialization Layer

Once the high-level intent is decomposed, the execution flow transitions to the Behavior Planning layer. This layer operates as a distributed ecosystem of specialized sub-agents that collaborate to map the Orchestrator’s generic functional intent into site-specific activation plans tailored for the IoT platform.

This translation and planning process is managed by two collaborative entities. First, the **Context Analysis Agent** acts as the primary analytical engine for the environment’s semantic data. It evaluates the current system constraints, resource availability, and operational boundaries necessary to fulfill the Orchestrator’s request. By establishing this contextual groundwork, the agent ensures that the proposed deployment logic remains valid and grounded within the rules of the target ecosystem.

Working in tandem is the **Platform Config Broker**. Receiving the validated context and the delegated functional tasks, this agent acts as the interoperability layer. It translates the agnostic descriptions into optimized, platform-specific configuration schemas, mapping the conceptual requirements to the exact API structures and data formats required by the target IoT environment.

4.4 Behavior Realization and Execution Layer

The final phase of the architecture is the Behavior Realization layer. This layer is responsible for the actual execution of the planned behaviors, translating the strategic and behavioral plans into concrete interactions with the target platform’s application interfaces.

To manage the diverse requirements of virtual execution, this layer is divided into three distinct, specialized agents.

Before any new deployment occurs, the **Configuration Reader** Agent queries the virtual IoT platform to assess the current state of the environment. It reads from the IoT platform active devices, and established data linkages. This ensures that new

deployments do not conflict with or overwrite critical existing infrastructure, providing the system with a vital baseline awareness.

At the same time, while the core deployment logic operates independently, specific scenarios may require external structural or specification data. The **Digital Twin Retrieval Agent** serves as a targeted retrieval engine, designed specifically to access and extract necessary informational parameters or architectural templates from external Digital Twin repositories when mandated by the deployment plan.

Finally, the **IoT Deployment Agent** operates as the primary actuation entity. It receives the fully validated, platform-specific configuration schemas from the broker and the baseline data from the reader agent, executing the final deployment. It interfaces directly with the IoT platform authenticating and issuing the API calls required to sequentially create device instances and apply the final configurations.

4.5 Platform and Services Layer

The underlying foundation of the entire system is the Platforms and Services layer, which acts as the data repository and target environment for the agentic loop.

4.6 Technologies used

Bibliography

- [1] Omer Ali et al. “A Comprehensive Review of Internet of Things: Technology Stack, Middlewares, and Fog/Edge Computing Interface”. en. In: *Sensors* 22.3 (Jan. 2022), p. 995. ISSN: 1424-8220. DOI: [10.3390/s22030995](https://doi.org/10.3390/s22030995). URL: <https://www.mdpi.com/1424-8220/22/3/995> (visited on 06/01/2026).
- [2] Jie Lin et al. “A Survey on Internet of Things: Architecture, Enabling Technologies, Security and Privacy, and Applications”. In: *IEEE Internet of Things Journal* 4.5 (Oct. 2017), pp. 1125–1142. ISSN: 2327-4662, 2372-2541. DOI: [10.1109/JIOT.2017.2683200](https://doi.org/10.1109/JIOT.2017.2683200). URL: <https://ieeexplore.ieee.org/document/7879243/> (visited on 06/01/2026).
- [3] Roberto Minerva, Gyu Myoung Lee, and Noel Crespi. “Digital Twin in the IoT Context: A Survey on Technical Features, Scenarios, and Architectural Models”. In: *Proceedings of the IEEE* 108.10 (Oct. 2020), pp. 1785–1824. ISSN: 0018-9219, 1558-2256. DOI: [10.1109/JPROC.2020.2998530](https://doi.org/10.1109/JPROC.2020.2998530). URL: <https://ieeexplore.ieee.org/document/9120192/> (visited on 06/01/2026).
- [4] Shuguo Hu et al. “Digital Twins Enabling Intelligent Manufacturing: From Methodology to Application”. en. In: *Intelligent and Sustainable Manufacturing* 1.1 (2024), pp. 10007–10007. ISSN: 3005-8066. DOI: [10.35534/ism.2024.10007](https://doi.org/10.35534/ism.2024.10007). URL: <https://www.sciepublish.com/article/pii/155> (visited on 06/01/2026).
- [5] Stefan Kopp et al. “Towards a common framework for multimodal generation: The behavior markup language”. In: *International workshop on intelligent virtual agents*. Springer. 2006, pp. 205–217.
- [6] Yuan He, Junchen Guo, and Xiaolong Zheng. “From Surveillance to Digital Twin: Challenges and Recent Advances of Signal Processing for Industrial Internet of Things”. In: *IEEE Signal Processing Magazine* 35 (5 Sept. 2018), pp. 120–129. ISSN: 1558-0792. DOI: [10.1109/MSP.2018.2842228](https://doi.org/10.1109/MSP.2018.2842228).
- [7] Jiasi Chen and Xukan Ran. “Deep Learning with Edge Computing: A Review”. In: *Proceedings of the IEEE* 107 (8 Aug. 2019), pp. 1655–1674. ISSN: 1558-2256. DOI: [10.1109/JPROC.2019.2921977](https://doi.org/10.1109/JPROC.2019.2921977).
- [8] Ala Al-Fuqaha et al. “Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications”. In: *IEEE Communications Surveys and Tutorials* 17 (4 Oct. 2015), pp. 2347–2376. ISSN: 1553-877X. DOI: [10.1109/COMST.2015.2444095](https://doi.org/10.1109/COMST.2015.2444095).
- [9] Michael Grieves. “Intelligent Digital Twins and the Development and Management of Complex Systems”. In: *Digital Twin* 2 (May 2022), p. 8. DOI: [10.12688/digitaltwin.17574.1](https://doi.org/10.12688/digitaltwin.17574.1).

- [10] Ahmed Awouda et al. “IoT-Based Framework for Digital Twins in the Industry 5.0 Era”. In: *Sensors* 24 (2 Jan. 2024). ISSN: 1424-8220. DOI: [10.3390/s24020594](https://doi.org/10.3390/s24020594).
- [11] Dirk Heylen et al. *AAMAS 2009 Workshop Towards a Standard Markup Language for Embodied Dialogue Acts PROCEEDINGS*. Tech. rep. 2009.
- [12] Ranjan Sapkota, Konstantinos I. Roumeliotis, and Manoj Karkee. “AI Agents vs. Agentic AI: A Conceptual taxonomy, applications and challenges”. en. In: *Information Fusion* 126 (Feb. 2026), p. 103599. ISSN: 15662535. DOI: [10.1016/j.inffus.2025.103599](https://doi.org/10.1016/j.inffus.2025.103599). URL: <https://linkinghub.elsevier.com/retrieve/pii/S1566253525006712> (visited on 05/04/2026).
- [13] Ashis Kumar Pati. “Agentic AI: A Comprehensive Survey of Technologies, Applications, and Societal Implications”. In: *IEEE Access* 13 (2025), pp. 151824–151837. ISSN: 2169-3536. DOI: [10.1109/ACCESS.2025.3585609](https://doi.org/10.1109/ACCESS.2025.3585609). URL: <https://ieeexplore.ieee.org/document/11071266/> (visited on 04/30/2026).
- [14] Ajay Bandi et al. “The Rise of Agentic AI: A Review of Definitions, Frameworks, Architectures, Applications, Evaluation Metrics, and Challenges”. en. In: *Future Internet* 17.9 (Sept. 2025), p. 404. ISSN: 1999-5903. DOI: [10.3390/fi17090404](https://doi.org/10.3390/fi17090404). URL: <https://www.mdpi.com/1999-5903/17/9/404> (visited on 05/03/2026).
- [15] Qiaomu Li and Ying Xie. “From Glue-Code to Protocols: A Critical Analysis of A2A and MCP Integration for Scalable Agent Systems”. en. In: ().
- [16] Abul Ehtesham et al. *A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)*. arXiv:2505.02279 [cs]. May 2025. DOI: [10.48550/arXiv.2505.02279](https://doi.org/10.48550/arXiv.2505.02279). URL: <http://arxiv.org/abs/2505.02279> (visited on 05/06/2026).
- [17] Aristeia Athanasopoulou, Nikos Fotiou, and Avraam Chatzopoulos. “Interacting with IoT Data Spaces Using LLMs and the Model Context Protocol”. en. In: *Sensors* 26.4 (Feb. 2026), p. 1193. ISSN: 1424-8220. DOI: [10.3390/s26041193](https://doi.org/10.3390/s26041193). URL: <https://www.mdpi.com/1424-8220/26/4/1193> (visited on 05/06/2026).
- [18] LangChain AI. *LangGraph: Building Stateful, Cyclic Multi-Agent Architectures*. Tech. rep. LangChain AI Research, 2024. URL: <https://github.com/langchain-ai/langgraph/>.
- [19] Harrison Chase. *LangChain: Core Data-Chaining Components and LLM Abstraction Frameworks*. Tech. rep. LangChain Inc., 2024. URL: <https://github.com/langchain-ai/langchain>.
- [20] Jerry Liu. *LlamaIndex: Unified Data Ingestion, Indexing, and Retrieval Frameworks for LLMs*. Tech. rep. LlamaIndex Inc., 2024. URL: https://github.com/run-llama/llama_index.
- [21] Microsoft Corporation. *Microsoft Agent Framework: Unified Enterprise AI Orchestration*. Tech. rep. Microsoft Learn Portal, 2025. URL: <https://learn.microsoft.com/en-us/azure/ai-services/>.
- [22] João Moura. *CrewAI: Orchestrating Role-Based Multi-Agent Systems*. Tech. rep. CrewAI Inc., 2024. URL: <https://docs.crewai.com>.

- [23] OpenAI Applied Research. *OpenAI Agents SDK: A Lightweight Framework for Multi-Agent Workflows*. GitHub Repository (openai-agents-python). 2026. URL: <https://github.com/openai/openai-agents-python>.
- [24] Flavio Corradini et al. “X-IoT: a model-driven approach to support IoT application portability across IoT platforms”. en. In: *Computing* 105.9 (Sept. 2023), pp. 1981–2005. ISSN: 0010-485X, 1436-5057. DOI: [10.1007/s00607-023-01155-z](https://doi.org/10.1007/s00607-023-01155-z). URL: <https://link.springer.com/10.1007/s00607-023-01155-z> (visited on 06/02/2026).
- [25] Fatima-Zohra Bemrah, Azeddine Chikh, and Samir Ouchani. “Hi-MAD: A Hierarchical Multi-Agent DRL framework for resilience and service optimization in IoT systems”. en. In: *Information and Software Technology* 196 (Aug. 2026), p. 108151. ISSN: 09505849. DOI: [10.1016/j.infsof.2026.108151](https://doi.org/10.1016/j.infsof.2026.108151). URL: <https://linkinghub.elsevier.com/retrieve/pii/S0950584926001400> (visited on 06/12/2026).
- [26] Jie Deng, Yongxu Zhu, and Fu Xiao. “Leveraging large language model agents for cost-effective sensor data handling and urban traffic navigation”. en. In: *Engineering Applications of Artificial Intelligence* 173 (June 2026), p. 114363. ISSN: 09521976. DOI: [10.1016/j.engappai.2026.114363](https://doi.org/10.1016/j.engappai.2026.114363). URL: <https://linkinghub.elsevier.com/retrieve/pii/S0952197626006445> (visited on 06/08/2026).
- [27] Chun-Chih Lin et al. “Achieving Robust Resource Orchestration for Highly Dense Heterogeneous IoT Systems”. In: *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*. London, United Kingdom: IEEE, May 2025, pp. 1–10. ISBN: 979-8-3315-4305-1. DOI: [10.1109/INFOCOM55648.2025.11044551](https://doi.org/10.1109/INFOCOM55648.2025.11044551). URL: <https://ieeexplore.ieee.org/document/11044551/> (visited on 06/15/2026).
- [28] Shuaishuai Guo et al. “Task-Oriented Communications for Agentic IoT: An LLM-Driven QoS/Security Policy Generation via Dynamic Model Context Protocol”. In: *2025 Seventeenth International Conference on Wireless Communications and Signal Processing (WCSP)*. Chongqing, China: IEEE, Oct. 2025, pp. 1–5. ISBN: 979-8-3315-8303-3. DOI: [10.1109/WCSP68525.2025.1010287](https://doi.org/10.1109/WCSP68525.2025.1010287). URL: <https://ieeexplore.ieee.org/document/11352215/> (visited on 06/26/2026).
- [29] David Carrascal et al. “Softwarized Edge Intelligence for Advanced IIoT Ecosystems: A Data-Driven Architecture Across the Cloud/Edge Continuum”. en. In: *Applied Sciences* 15.19 (Oct. 2025), p. 10829. ISSN: 2076-3417. DOI: [10.3390/app151910829](https://doi.org/10.3390/app151910829). URL: <https://www.mdpi.com/2076-3417/15/19/10829> (visited on 06/25/2026).